

Enumeradores (ENUM)

Representando um conjunto fixo de valores

Contexto: um conjunto fixo de valores

Várias situações do dia a dia envolvem um conjunto **fixo e conhecido** de valores possíveis:

- os dias da semana;
- os níveis de dificuldade de um jogo (fácil, médio, difícil);
- o status de um pedido (pendente, pago, cancelado).

Em todos esses casos, **não existe qualquer outro valor possível** além desses – não existe um "oitavo dia da semana".

Uma tentativa comum (mas problemática)

Um jeito de representar isso, antes de conhecer enums, é usar constantes numéricas:

```
public class DiaDaSemana {  
    public static final int DOMINGO = 0;  
    public static final int SEGUNDA = 1;  
    public static final int TERCA = 2;  
    // ...  
}  
  
public void agendarReuniao(int diaSemana) {  
    if (diaSemana == DiaDaSemana.SEGUNDA) {  
        System.out.println("Reunião marcada para segunda");  
    }  
}
```

Os problemas dessa abordagem

```
agendarReuniao(DiaDaSemana.SEGUNDA); // ok
agendarReuniao(99);                    // compila, mas 99 não
                                        // é um dia válido!
agendarReuniao(-5);                    // também compila...
```

O problema

O parâmetro é apenas um `int`. O compilador **não tem como saber** que só os valores de 0 a 6 fazem sentido – qualquer número inteiro é aceito.

O mesmo problema aconteceria se usássemos `String`: nada impede passar "segundaa" (com erro de digitação) no lugar de "segunda".

A solução do Java: enum

Definição

Um **enum** (tipo enumerado) é um tipo especial em Java usado para representar **um conjunto fixo e conhecido de constantes**.

- cada valor possível vira uma **constante nomeada** do próprio tipo;
- o compilador **só aceita** os valores declarados no enum – nada mais.

Sintaxe: declarando um enum simples

Trocamos a palavra-chave `class` por `enum`, e listamos as constantes separadas por vírgula:

```
public enum DiaDaSemana {  
    DOMINGO, SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA, SABADO;  
}
```

- cada palavra em maiúsculo (DOMINGO, SEGUNDA, ...) é uma **constante** do tipo `DiaDaSemana`;
- por convenção, o nome das constantes de um `enum` é escrito em MAIÚSCULO.

Usando o enum no Main

```
public class Main {  
    public static void main(String[] args) {  
        DiaDaSemana dia = DiaDaSemana.SEGUNDA;  
  
        System.out.println(dia); // SEGUNDA  
    }  
}
```

- o tipo da variável é DiaDaSemana;
- o valor atribuído é uma das constantes declaradas no enum, acessada como DiaDaSemana.SEGUNDA.

A grande vantagem: segurança de tipos

```
DiaDaSemana dia = DiaDaSemana.SEGUNDA; // ok  
  
// DiaDaSemana dia2 = DiaDaSemana.SEGUNDA; // ERRO DE COMPILAÇÃO!  
// DiaDaSemana dia3 = 99; // ERRO DE COMPILAÇÃO!
```

Diferença em relação ao int ou String

Com um enum, o **compilador** verifica se o valor realmente existe. Não é mais possível passar um número inválido ou um texto com erro de digitação – o erro aparece **antes mesmo do programa rodar**.

Métodos prontos de todo enum

Todo enum em Java já vem com alguns métodos úteis, sem precisar escrever nada:

```
DiaDaSemana dia = DiaDaSemana.QUARTA;

System.out.println(dia.name());      // "QUARTA"
System.out.println(dia.ordinal());   // 3 (posição na lista, começa
                                     em 0)

for (DiaDaSemana d : DiaDaSemana.values()) {
    System.out.println(d); // imprime todas as constantes
}

DiaDaSemana outro = DiaDaSemana.valueOf("SEXTA");
System.out.println(outro); // SEXTA
```

Enum funciona muito bem com switch

```
DiaDaSemana dia = DiaDaSemana.SABADO;

switch (dia) {
    case SABADO:
    case DOMINGO:
        System.out.println("Fim de semana!");
        break;
    default:
        System.out.println("Dia de semana normal");
}
```

Repare que, dentro do switch, usamos apenas SABADO e DOMINGO, sem precisar escrever `DiaDaSemana.SABADO` – o Java já sabe o tipo pelo contexto.

E se cada constante precisar guardar mais informação?

Nem sempre um nome é suficiente. Pense no exemplo do nível de dificuldade de um jogo:

- **FÁCIL** – multiplica o XP ganho por 1.0;
- **MÉDIO** – multiplica o XP ganho por 1.5;
- **DIFÍCIL** – multiplica o XP ganho por 2.0.

Pergunta

Como fazer cada constante do enum **carregar um valor próprio**, além do nome?

Um enum pode ter atributos, construtor e métodos

A ideia

Um enum é, por baixo dos panos, **uma classe especial**. Assim como uma classe comum, ele pode ter atributos, um construtor e métodos – inclusive get.

- cada constante passa a **chamar o construtor**, como se estivesse criando um objeto;
- os valores passados ficam guardados em atributos privados de cada constante.

Código: enum NivelDificuldade

```
public enum NivelDificuldade {  
    FACIL(1.0),  
    MEDIO(1.5),  
    DIFICIL(2.0);  
  
    private final double multiplicador;  
  
    private NivelDificuldade(double multiplicador) {  
        this.multiplicador = multiplicador;  
    }  
  
    public double getMultiplicador() {  
        return multiplicador;  
    }  
}
```

Cada constante (FACIL, MEDIO, DIFICIL) chama o construtor com o seu próprio valor de multiplicador.

Por que o construtor é sempre privado?

- as únicas instâncias que podem existir de um enum são **as constantes declaradas no topo** (FACIL, MEDIO, DIFICIL);
- por isso, **ninguém de fora** pode criar uma nova instância com `new NivelDificuldade(...)` – nem o próprio enum permite.

Detalhe importante

Mesmo se você não escrever a palavra `private`, o construtor de um enum já é **privado automaticamente**. Escrever `private` explicitamente é apenas uma forma de deixar isso claro para quem lê o código.

Usando o NivelDificuldade no Main

```
public class Main {  
    public static void main(String[] args) {  
        NivelDificuldade nivel = NivelDificuldade.MEDIO;  
  
        System.out.println(nivel.getMultiplicador()); // 1.5  
  
        for (NivelDificuldade n : NivelDificuldade.values()) {  
            System.out.println(n.name()  
                + " -> multiplicador " + n.getMultiplicador());  
        }  
    }  
}
```

Cada constante se comporta como um pequeno objeto, com seu próprio valor guardado no atributo multiplicador.

Lendo a dificuldade escolhida pelo jogador

Até agora, sempre escrevemos o nível direto no código:

```
NivelDificuldade nivel = NivelDificuldade.MEDIO;
```

Mas, em um jogo de verdade, quem escolhe a dificuldade é o **jogador**, digitando no teclado.

Pergunta

Como transformar um texto digitado pelo jogador (por exemplo, "MEDIO") em uma constante do tipo `NivelDificuldade`?

Passo 1: lendo o texto com Scanner

```
Scanner sc = new Scanner(System.in);  
  
System.out.print("Escolha o nível (FACIL, MEDIO, DIFICIL): ");  
String texto = sc.nextLine();
```

- o Scanner lê uma linha digitada pelo usuário;
- o resultado é apenas uma String comum – o Java ainda **não sabe** que esse texto deveria virar um NivelDificuldade.

Passo 2: convertendo o texto com valueOf()

```
Scanner sc = new Scanner(System.in);

System.out.print("Escolha o nível (FACIL, MEDIO, DIFICIL): ");
String texto = sc.nextLine();

NivelDificuldade nivel =
    NivelDificuldade.valueOf(texto.toUpperCase());

System.out.println("Multiplicador: " + nivel.getMultiplicador());
```

Por que toUpperCase()?

valueOf() exige o texto **idêntico** ao nome da constante. Se o jogador digitar "medio" em minúsculo, toUpperCase() transforma em "MEDIO" antes da comparação.

Cuidado: entrada inválida quebra o programa

E se o jogador digitar errado?

Se o jogador digitar "facim" ou "impossivel", nenhuma constante de `NivelDificuldade` tem esse nome.

O que acontece

`valueOf()` lança uma exceção (`IllegalArgumentException`) e o programa **para de executar**, caso ninguém trate esse erro.

Precisamos tratar esse caso, assim como tratamos qualquer outra entrada inválida do usuário.

Passo 3: tratando entradas inválidas

```
Scanner sc = new Scanner(System.in);
System.out.print("Escolha o nível (FACIL, MEDIO, DIFICIL): ");
String texto = sc.nextLine();

try {
    NivelDificuldade nivel =
        NivelDificuldade.valueOf(texto.toUpperCase());

    System.out.println("Multiplicador: "
        + nivel.getMultiplicador());
} catch (IllegalArgumentException e) {
    System.out.println("Nível inválido! Tente FACIL, "
        + "MEDIO ou DIFICIL.");
}
```

Com o try/catch, uma entrada inválida vira uma **mensagem amigável**, em vez de derrubar o programa.

Testando enums com JUnit

Um enum com atributos e métodos é, por dentro, uma classe – então **pode e deve ser testado** como qualquer outra.

- podemos comparar constantes diretamente com `assertEquals`;
- podemos verificar se o valor de um atributo (`getMultiplicador()`) está correto;
- podemos percorrer `values()` para testar **todas** as constantes de uma vez.

Teste 1: comparando com valueOf()

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class NivelDificuldadeTest {

    @Test
    void deveConverterTextoParaConstanteCorreta() {
        NivelDificuldade nivel =
            NivelDificuldade.valueOf("DIFICIL");

        assertEquals(NivelDificuldade.DIFICIL, nivel);
    }
}
```

Comparar enums com assertEquals funciona normalmente, assim como comparar com ==.

Teste 2: verificando os atributos

```
@Test
void facilDeveTerMultiplicadorUm() {
    assertEquals(1.0, NivelDificuldade.FACIL.getMultiplicador());
}

@Test
void dificilDeveSerMaiorQueMedio() {
    double dificil = NivelDificuldade.DIFICIL.getMultiplicador();
    double medio = NivelDificuldade.MEDIO.getMultiplicador();

    assertTrue(dificil > medio);
}
```

Teste 3: percorrendo todas as constantes

```
@Test
void todosOsNiveisDevemTerMultiplicadorPositivo() {
    for (NivelDificuldade nivel : NivelDificuldade.values()) {
        assertTrue(nivel.getMultiplicador() > 0);
    }
}
```

Vantagem

Esse teste continua funcionando mesmo se, no futuro, adicionarmos um novo nível de dificuldade ao enum – ele será testado automaticamente, sem precisar alterar o teste.

Recapitulando

- `enum` representa **um conjunto fixo e conhecido** de constantes, com segurança de tipos garantida pelo compilador;
- todo `enum` já vem com métodos prontos: `values()`, `name()`, `ordinal()` e `valueOf()`;
- um `enum` pode ter **atributos, construtor e métodos**, assim como uma classe comum;
- o construtor de um `enum` é **sempre privado** – as únicas instâncias possíveis são as constantes declaradas;
- um `enum` pode ser usado e testado normalmente, tanto no `Main` quanto em testes `JUnit`.